

Pré-processamento de Dados

Hoop Vision

Aprendizado de máquinas

Profa. Jaqueline Pugliesi

FATEC Franca - Desenvolvimento de Software Multiplataforma 5º
Semestre

Bruno Algarte Inácio
Cristian Tulio Garcia Do Nascimento
Eduardo Vilas Boas Freitas
Rafael Verrísimo da Silva

1º Passo: Divisão dos Dados em Treinamento e Teste

Após o pré-processamento, realizamos a divisão da base entre **dados de treinamento (80%)** e **teste (20%)**, com estratificação para manter a proporção das classes:

```
from sklearn.model_selection import train_test_split

X = df[['Pos', 'PPG', 'APG', 'RPG']]
y = df['Label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

2º Passo: Transformação de Dados Categóricos

A variável **Pos** foi transformada em variáveis numéricas por meio do OneHotEncoder:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocessador = ColumnTransformer(
    transformers=[('onehot', OneHotEncoder(drop='first'), ['Pos'])],
    remainder='passthrough'
)
```

3º Passo: Avaliação com Múltiplos Algoritmos Supervisionados

Realizamos testes com diversos algoritmos supervisionados para identificar o que melhor se adaptava à nossa base de dados. Abaixo os principais resultados:

Algoritmo	Acurácia	Superstar (Precision)	Superstar (Recall)	Observações
Random Forest	99%	0.90	0.79	Melhor desempenho geral. Escolhido como modelo final.
SVM	97%	0.94	0.67	Alta precisão, mas menor recall para Superstar.
Regressão Logística	96%	0.88	0.58	Performance consistente, mas inferior ao Random Forest
Árvore de Decisão	97%	0.91	0.88	Ótimo recall, mas menor robustez em comparação.
Naive Bayes	32%	1.00	0.33	Performance geral ruim, descartado.

Cada modelo foi avaliado com métricas como **precisão**, **recall**, **f1-score** e **acurácia total**, utilizando a biblioteca `scikit-learn`.

4º Passo: Escolha e Treinamento Final

Com base na análise das métricas, o algoritmo **Random Forest Classifier** foi escolhido como o classificador final, devido à sua alta robustez e desempenho superior mesmo para a classe minoritária "Superstar".

Código completo de treinamento: `modelo_treino_RandomForest.py`

5º Passo: Exportação do Modelo Treinado

O modelo foi salvo utilizando a biblioteca `joblib`, permitindo sua integração com o backend da aplicação.

Conclusão

O pós-processamento consolidou todo o ciclo de aprendizado supervisionado, com:

- Testes com múltiplos algoritmos

- Escolha baseada em desempenho real
- Modelo salvo e pronto para produção

O modelo treinado agora é consumido pelo backend da aplicação e exposto ao usuário no aplicativo mobile, concluindo assim a jornada da inteligência artificial dentro do projeto **Hoop Vision**.